

Continnum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live

Hanchen Li*
UC Berkeley
Berkeley, California, United States
lihanc@berkeley.edu

Runyuan He*
UC Berkeley
Berkeley, California, United States
runyuan@berkeley.edu

Qiuyang Mang*
UC Berkeley
Berkeley, California, United States
qmang@berkeley.edu

Qizheng Zhang
Stanford University
Stanford, California, United States
qizhengz@stanford.edu

Huanzhi Mao
UC Berkeley
Berkeley, California, United States
huanzhimao@berkeley.edu

Xiaokun Chen
Stanford University
Stanford, California, United States
xiaokunc@stanford.edu

Hangrui Zhou
Tsinghua University
Beijing, China
zhouhr23@mails.tsinghua.edu.cn

Huanchen Zhang
Tsinghua University
Beijing, China
huanchen@tsinghua.edu.cn

Alvin Cheung
UC Berkeley
Berkeley, California, United States
akcheung@cs.berkeley.edu

Joseph Gonzalez
UC Berkeley
Berkeley, California, United States
jegonzal@berkeley.edu

Ion Stoica
UC Berkeley
Berkeley, California, United States
istoica@berkeley.edu

ABSTRACT

KV cache management is essential for efficient LLM inference. To maximize utilization, existing inference engines evict finished requests' KV cache if new requests are waiting. This policy breaks for agentic workloads, which interleave LLM calls with tools, introducing pauses that prevent effective KV reuse across turns. Since many tool calls have much shorter durations than human response multi-turn chatbot, it would be promising to retain the KV cache in during these tools. However, many challenges remain. First, we need to consider both the potential cost of recomputation or reloading (if offloading enabled) as well as the increasing queueing delays after eviction from GPU. Second, due to the internal variance of tool call durations, the method needs to remain robust under limited predictability of tool call durations.

We present Continnum, a serving system to optimize job completion time for multi-turn agent workloads by introducing time-to-live mechanism for KV cache retention. For requests that generate tool calls, Continnum selectively pins the KV cache in GPU memory with a time-to-live value determined by the reload cost and potential queueing delay induced by eviction. When the TTL expires, the KV cache can be automatically evicted to free up GPU memory, providing robust performance under edge cases. When combined with program-level first-come-first-serve, Continnum preserves multi-turn continuity, and reduces delay for agentic workflows. Evaluations on real-world agents (SWE-Bench, BFCL, OpenHand) with Llama-3.1 8B/70B, Gemma-3 12B, and GLM-4.5 355B shows that Continnum improves the average job completion times by over 8x while improving throughput.

PVLDB Reference Format:

*The first three authors contributed equally to this work.

Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continnum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

1 INTRODUCTION

KV Cache management is key to large language model inference, impacting both the input processing (prefill) and output generation (decoding) stages [14, 28, 66]. A critical component of KV cache management is the eviction policy. Ideally, the system should avoid evicting tokens that will be referenced in the immediate future. Similar to traditional caching systems, Existing inference engines assume that KV caches are less important once decoding is finished. To maximize utilization, these engines discard the KV caches of completed requests when new requests are waiting in the queue. We refer to this type of policy **end-of-turn eviction**.

While end-of-turn eviction works well for multi-turn chat applications, it can significantly degrade the performance of modern agentic workloads, particularly those involving tool calling. These agentic applications have become increasingly popular across domains such as software engineering [57], computer use [7], and scientific research [45]. These workloads characteristically interleave (a) inference steps to derive the next action, and (b) execution steps where the agent calls an external tool. The output of the tool is

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

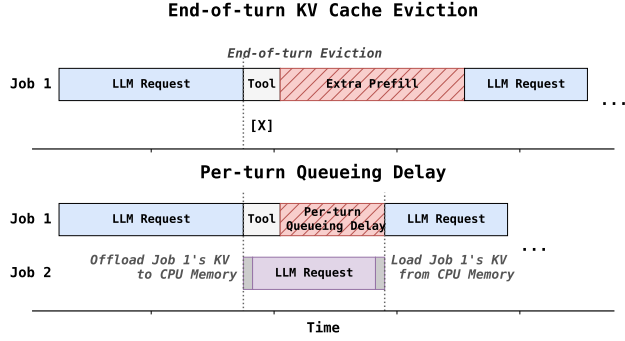


Figure 1: Two main failure modes of prior agent-serving systems. Red blocks represent overhead from suboptimal scheduling and KV-cache management: even with CPU offloading, agents still suffer queueing delay after KV-cache eviction.

subsequently appended to the request context, and a new inference step is initiated in the inference engine. Since the tool call can be much faster (*i.e.*, $\leq 2s$) than human typing speed, this new workload requires changes to end-of-turn eviction.

The core issue arises after the request’s KV cache is evicted when the agent transforms from inference step to tool call. If the KV cache was evicted for this step, the engine must recompute the prefix (prefill) or reload from CPU (if CPU offloading is enabled [14]) when the tool execution completes and the next inference step begins. This repetitive prefill introduces substantial delays and reduces overall system throughput. More importantly, even when CPU offloading is enabled to reuse KV cache, eviction causes another problem: **per-turn queueing delay**. When the next inference step has its KV cache evicted from GPU memory, even if the KV cache can be reloaded from CPU, it will also have to wait in the waiting queue for other requests to free up GPU memory before starting inference. This per-turn queueing delay can accumulate and result in increasing delay for each agentic program as illustrated in Figure 1. Since this delay is not measurable by offline profiling, we need to design a new model to include its impact. Moreover, since tool calls can be inherently variable, we need to set a maximum KV cache retention time to prevent infinitely long waiting. However, if this time expires just before the tool call, the previous waiting time will be wasted. Thus, we need to carefully set the KV cache retention time to best adapt to the workload.

Previous work fails to address these challenges. InferCept [2] makes its KV preserve decision based solely on the reload cost. But it does not model the per-turn queueing delay that accumulates over turns, nor have a robust mechanism to handle variable tool call durations. This makes it impractical for real-world deployment. As we show later in Section 6, InferCept accumulates the queueing penalty over turns, resulting in suboptimal performance. Autelix [39] uses end-of-turn eviction and ignores the importance of KV cache retention in multi-turn agent scheduling. Pie [20] exposes interfaces but provides no policy for KV cache retention decisions. Ayo [49], Alto [46], and Parrot [34] assume static workflows and do not apply to dynamic agents.

To provide an efficient and robust solution, we present Continnum, a serving system that utilizes KV cache time-to-live technique to improve job completion time for multi-turn agent workloads. Inspired by previous caching papers, Continnum introduces a KV cache time-to-live (TTL) mechanism to retain KV cache inside GPU after request finishes to over-ride original end-of-turn evictions. For each LLM request that generates a tool call during the inference step, Continnum models both the prefill/reload cost and the per-turn queueing delay reduction brought retaining KV cache. After obtaining the benefits of a potential hit based on the above two factors and tool call distributions, Continnum compares this with the cost of occupying GPU memory space during the TTL time to decide how long the KV cache can stay in GPU memory before being automatically evicted. This allows the next request to immediately resume if the tool call returns within the TTL window to save prefill and queueing delay. When the tool call prediction is inaccurate and the tool call takes longer than expected, Continnum can correct the mistake robustly by evicting the KV cache after the TTL expires, preventing severe memory pressure or deadlocks. Furthermore, Continnum combines the TTL mechanism with program-level first-come-first-serve scheduling. This enforces better request ordering and simplifies scheduling for complex agentic workflows.

We implemented Continnum on top of vLLM with a modular design that can be easily maintained or integrated into other inference engines. Continnum implemented a tool call handler that is called each time a request enters or leaves the serving engine. It identifies the tool call, predicts the duration, and decides the timeout of the KV cache pin based on both throughput and request ordering concerns. This modular design adds minimal change to the original scheduling logic of the inference engine and allows for future extension to tool-call aware scheduling.

To evaluate Continnum’s performance, we conduct extensive experiment on real agentic workloads in function calling [41] and coding agents [33]. Across three hardware and model setups, we show that Continnum reduces delay by 1.12x to 3.66x and improves throughput by 1.10x to 3.22x on multi-turn agentic workloads. Moreover, we evaluated Continnum on Company A’s¹ internal testbed and show it can reduce delay for real SWE-agent workloads by up to 8.18x. We will open-source our traces, code, and the agent serving testbed to foster future agent serving research.

In summary, our contributions are the following:

- We identify the key cache KV retention problem in agent serving and motivates the need for better solution.
- We design Continnum, a efficient and robust serving system with KV cache time-to-live mechanism to reduce turn-based eviction cost and per-turn queueing delay.
- We demonstrate that Continnum achieves up to 8.18x improvements in both latency and throughput over previous methods in both emulated and real cases.
- We will open-source our collected agent inference traces, code, and agent serving testbed upon publication.

¹Inference startup anonymized for double-blind

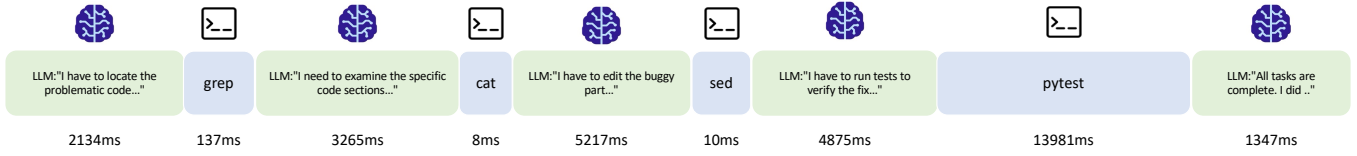


Figure 2: Illustrative example of a SWE-Agent. The agent resolves a software engineering bug step by step with tool calls in the middle. These tool calls have different durations and breaks the continuity of the LLM inference.

2 BACKGROUND

2.1 ReAct Paradigm for Agents

Most modern agentic workflows follow the *ReAct*-agent loop [61], alternating between a reasoning step where the LLM interprets context and outputs thoughts, and an action step where it invokes external tools. This paradigm has become the de facto standard: coding agents such as Claude Code [8] and Cursor [16] adopt it for its clarity and performance, frameworks like LangChain [30] and LangGraph [31] make the pattern broadly accessible, and recent open-weight models including GPT-OSS [3] and Kimi-K2 [1] bake tool-call ability directly into the base model.

An important trend is that agentic applications increasingly scale this loop into *long-horizon, multi-turn* iterations, repeatedly interleaving thought, tool call, and context update across dozens or even hundreds of turns. This is reflected in recent benchmarks such as τ -bench for tool-agent-user interaction [60], MINT for multi-turn tool-augmented interaction [52], and AgentBench for multi-turn decision-making and tool-use scenarios [36].

2.2 Limitations of Existing Methods

Previous works fail to handle this emerging complex workload due to three main reasons:

Fixed Workflow: One line of work focused on scheduling agentic workflows with **pre-defined, static** computation graphs. Teola [49] decomposes applications into primitive-level dataflow graphs and then applies graph-level optimizations. Alto [46] focuses on streaming and pipelined execution across distributed components. Parrot [34] exposes application-level context to LLM services through Semantic Variables, enabling the engine to infer data dependencies across consecutive LLM requests. One shared limitation of Teola, Parrot, and Alto is that they all assume static or deterministically defined DAGs and **could not work with dynamic agent workloads** like ReAct-styled ones whose dependency graphs evolve at runtime. This limits these work from optimizing for the wide variety of agents in practice [9, 33, 56].

No Tool-Call Awareness: Wadlom et al. [50] study efficient LLM serving for agentic workflows from a data-systems perspective. Autellix [39] introduces Program-Level Attained Service (PLAS) scheduling that prioritizes requests with less cumulative service time of the agentic program. Tempo [65] proposes a scheduler to satisfy the SLOs when facing different types of requests (chat, agent, reasoning), while our focus is particularly on agentic workloads with many-turn and variable tool calls. Apt-Serve [19] jointly optimizes hybrid-cache management and request scheduling to improve effective throughput, but it targets individual LLM requests rather

Table 1: Continuum comparison with representative baselines.

Method	Retains KV Cache	Includes Per-Turn Queueing Delay	Bounds Retention Time
vLLM	✗	✗	✗
Autellix	✗	✗	✗
Pie	✓	✗	✗
InferCept	✓	✗	✗
Continuum	✓	✓	✓

than dynamic multi-turn agent workflows. AlignedServe explores prefix-aware batching for LLM serving [10]. These work fail to consider the unique characteristics of tool calls in agentic workloads, such as their variable durations and the impact on KV cache management. This oversight can lead to suboptimal scheduling decisions and increased latency, as we demonstrate later in Sec 3.2.

Insufficient KV Cache Retention Strategies: Some previous work observed the challenge of KV cache reuse for agent workloads. Cache-Craft [4] manages reusable chunk caches for efficient retrieval-augmented generation, but it focuses on document chunks rather than retaining request-specific agent state across tool calls. InferCept [2] introduces a “preserve” operation that pins the KV cache between tool calls. However, their policy overlooks the multi-turn nature of requests. When KV cache is evicted between turn, this will cause additional queueing time per turn for the program when they come back. In multi-turn scenarios, the queueing time can accumulate for each turn. Ignoring such effects makes them not preserve KV cache in GPU even when there are significant benefits. Moreover, their preserve operation is fixed and could not adapt to tool use in real time. If the actual tool call time is much longer than predicted, blindly “preserving” the KV cache can cause significant inefficiency. This makes it impractical for real-world deployment. Pie [20] introduces a programmable serving system that decomposes the generation loop into fine-grained handlers. It delegates control to user programs, allowing for custom tool call handling. However, it requires developers to manually design scheduling for each agent, and provides no actual method to adapt to dynamic tool-call latencies or multi-turn dependencies.

3 MOTIVATION

3.1 Agentic Traces

We begin by analyzing the characteristics of modern agentic workloads. We collect and analyze 100 mini-swe-agent traces [33] running SWE-Bench [23]. We also collect 100 traces from BFCL V4 Web

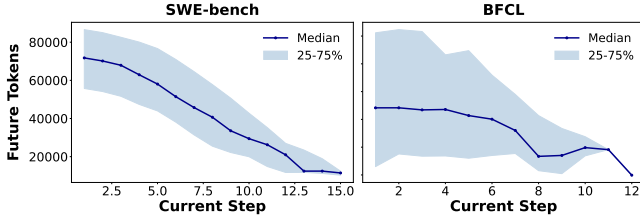


Figure 3: Workload characteristics of agentic workloads SWE-Bench and BFCL as used in Sec 6. As the number of steps increase, the requests are closer to finish.

Table 2: Statistics from two collected datasets. Reported numbers are in the format of (mean, standard deviation).

Dataset	No. of Turns	Tool Time(ms)	Token Per Program
SWE-Bench	(10.9, 2.1)	(925, 3,550)	(70,126, 19,732)
BFCL v4	(6.3, 2.3)	(1,923, 2,133)	(93,256, 68,687)

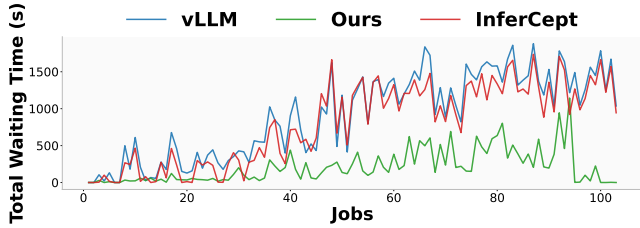


Figure 4: Per-program queueing delay under CPU offloading. InferCept’s preserve decision ignores queueing cost, so evicted programs still accumulate substantial waiting time across turns—comparable to vanilla vLLM despite InferCept’s reload savings.

Search [40]; both workloads use GPT-5 as the base model. Figure 2 presents an illustrative shortened example trace from SWE-Bench, demonstrating how the agent solves a software engineering task step by step.

The takeaway is three-fold. First, there are many turns for these novel agentic programs. This increase in turn numbers adds additional scheduling difficulty. Second, the tool call times have a varying distribution, but many are short. Although the request will be considered finished after these short tool calls are generated, the next request will arrive soon after the tool call completes, reusing the KV cache.

Last but not least, as shown in Figure 3, as the program approaches completion, the expected number of future tokens overall reduces for both workloads. This indicates that later turns have shorter expected finish time. This suggests that prioritizing requests that came earlier (program-level FCFS) or have executed more turns could be a good approximation for the theoretically optimal but clairvoyant shortest remaining time first (SRTF) scheduling policy. But it is non-trivial to maintain such ordering when tool calls are involved, as we will discuss later in 3.2.

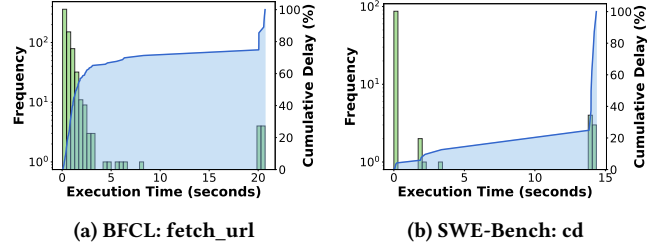


Figure 5: Functions’ execution time can be extremely long-tailed. Slowest 10% of fetch_url account for 52.5% of the total delay, while slowest 10% of cd account for 94.1%.

3.2 Challenges for Agentic Workloads

Turn-based Eviction: Although these tool calls can be short, inference engines treat them as homogeneous gaps between LLM requests. vLLM or SGLang will evict a request’s KV cache as soon as decoding finishes, implicitly assuming the request is complete. However, if the KV cache has been evicted, the engine must either redo the full prefill or reload the KV cache from DRAM when offloading is enabled, incurring additional delay. Most systems fall short in handling these scenarios efficiently.

Figure 1 illustrates this effect: the tool call creates a pause that triggers KV cache eviction, leading to prefill or KV reload on return. Thus, it is important to have a KV cache retention policy that considers tool calls to avoid such overheads.

Per-Turn Queueing Delay: The multi-turn nature agent programs also introduces a new challenge for scheduler that prior work have critically overlooked. While the current agent program is waiting on the tool, if the scheduler allocates the GPU memory to other requests to maximize throughput, the KV cache for the current program will be removed from GPU memory. When the program’s tool call returns and the following LLM request is sent to the scheduler, it must wait behind ongoing prefill/decoding of other requests for free GPU space.

This waiting period produces a gap in the execution of the agent program regardless whether the KV cache is stored in a CPU DRAM location. As shown by Figure 1, this gap also contributes to the delay induced by the tool call besides the previous prefill/loading cost, accumulating over turns and causing substantial delays for each program. Moreover, it also breaks the continuity of the program execution and schedules requests with earlier arrival times after later ones. Notice that even if we give the highest priority to the new request in the waiting queue, it still will be blocked by the ongoing computation of the other requests already in GPU.

Existing works do not consider per-turn queueing delay in their retention policies. InferCept [2]’s KV “preserve” operation is invoked only when the CPU offloading cost exceeds the estimated GPU occupation cost during the tool call. Crucially, this decision only accounts for the *reload cost* of the immediate next turn—it entirely ignores the queueing delay that an evicted program will experience when it re-enters the waiting queue behind other active requests. With fast asynchronous CPU offloading provided by

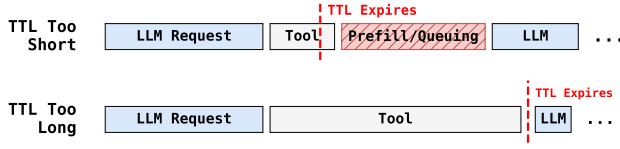


Figure 6: Time-to-live needs to be well set to balance between memory usage and the prefill plus per-turn queueing delay.

engines like LMCache [14], the reload cost becomes small, so InferCept’s preserve operation is rarely invoked. Yet the queueing delay persists regardless of offloading speed: even with instant KV reload, the returning request must still wait for GPU memory occupied by other requests to be freed. Since this queueing cost is incurred at *every* turn, the total accumulated delay grows proportionally with the number of turns per program—precisely the regime where agentic workloads operate.

We demonstrate the performance degradation brought by this lack of consideration for multi-turn scheduling in Figure 4. We profile the total eviction overhead experienced by each request for vanilla vLLM and the InferCept algorithm. The x-axis represents each agentic program in order of arrival time, while the y-axis denotes the total bubble time for each agentic job — the total idle period a request experiences in the waiting queue before execution. Even with InferCept’s KV retention, bubbles still persist and causes delay increase despite its throughput improvement over vLLM.

Variable Tool Call: Current KV cache retention policy also fail under greatly varying tool calls. For example, InferCept’s approach pins the KV cache in GPU memory until the next request arrives after a tool call. This methods works fine under stable tool call latencies. However, as shown in Figure 5, many tool calls exhibit high variability in execution time. When the tool call takes much longer than expected, the pinned KV cache could occupy GPU memory for a long time. Similar patterns are observed in database agents, as external tool calls are more complex. This leads to inefficient memory usage and even potential deadlocks when retained KV cache fully occupies the GPU. Thus, a static retention policy lacks robustness in practical scenarios.

4 CONTINNUM SCHEDULING ALGORITHM

Given the failure of previous work, we identify the key question in serving agentic workloads: How to efficiently and robustly retain KV cache in multi-turn scenarios?

An optimal KV cache retention policy should include the following features:

- It should retain KV cache for requests that will reuse them soon after tool calls, minimizing prefill/loading overheads.
- It should consider the multi-turn continuity of agent programs, reducing waiting and preserving program order.
- It should be robust to varying tool call latencies.

In order to achieve the robustness guarantee, we propose to borrow the idea of Time-to-live (TTL) from traditional systems: for each request’s KV cache, we give a TTL value to define the maximum

Algorithm 1: Continnum’s Scheduling Algorithm

Global state: waiting queue Q ; TTL map P (records pinned programs and their TTLs); historical tool-call records S , where $S[f]$ denotes the recorded tool-call information for tool f

```

1 Function OnRequestArrive(request  $r$ ):
2    $Q \leftarrow Q \cup \{r\}$ ,  $id \leftarrow$  Program ID of  $r$ ;
3   If  $id$  is a seen program then
4      $(f, t) \leftarrow$  Tool-call information from  $r$ ;
5      $S[f] \leftarrow S[f] \cup \{t\}$ ;
6 Function OnRequestFinish(request  $r$ ):
7   If  $r$  is the last request of its program then
8     Free KV cache used by  $r$ ;
9   else
10     $f \leftarrow$  Next tool to be called after finishing  $r$ ;
11     $id \leftarrow$  Program ID of  $r$ ;
12     $P[id] \leftarrow \text{CalcTTL}(r, S[f])$ ;
13 Function Schedule():
14   While  $Q$  is not empty do
15     For each  $id$  in  $P$ .keys do
16       If current time  $> P[id]$  and  $id \notin Q$ .programs then
17         Free KV cache used by  $id$ ’s last request;
18          $P \leftarrow P \setminus (id, P[id])$ ;
19      $r \leftarrow \text{argmax}_{r' \in Q} \text{CalcPriority}(r', P)$ ;
20     If  $r$  cannot fit into memory then
21       break;
22     else
23        $Q \leftarrow Q \setminus \{r\}$ ;
24       Issue  $r$  to running;
25        $id \leftarrow$  Program ID of  $r$ ;
26       If  $id \in P$ .keys then  $P \leftarrow P \setminus (id, P[id])$ ;

```

duration for it to remain in GPU memory. This prevents long-running or failed tool calls from blocking GPU resources indefinitely while retaining KV cache.

However, setting appropriate TTL values for each KV cache entry is challenging compared with static preserve operations. First, the TTL value should not be too large. If the timeout duration is too long as shown in Figure 6, the pinned KV cache occupies GPU memory unnecessarily, blocking other requests and reducing overall system throughput. On the other hand, if the pin time for the specific KV cache is too short, the KV cache is evicted before the tool call completes, still causing expensive recomputation or scheduling bubble despite wasted GPU occupation time.

Given these tradeoffs, the TTL value should be set carefully. Only if we can set appropriate TTL values based on based on tool call durations, prefill/loading costs, and the measurement to program continuity, we can balance the benefit of cache reuse against the need to maintain system throughput for other requests to achieve good performance.

Table 3: Key notations in Continnum’s cost model for a request r and its associated tool-call f .

Notation	Description
τ	TTL
$\text{MemUsage}(r)$	GPU memory occupied by r
$\mathcal{M}$	Average memory occupied by the seen requests
$\text{CacheMissCost}(r)$	Cost of reloading r
$\text{Prefill-Reload}(r)$	Time for reconstructing KV cache in GPU
$\text{OutOfOrderCost}(r)$	Cost of out-of-order for request r
η	Memoryfulness factor of the workload
$\mathcal{T}$	Average waiting time
$\mathcal{P}(\tau, f)$	Estimated finish-within-TTL probability for f

4.1 Utility Model

To set an effective TTL value (in seconds) for pinning a request’s KV cache, Continnum must choose the value that best balances the benefit of potential reuse against its cost. Both the benefit and the cost are measured in units of time, since they ultimately translate into changes in the total job completion latency across all programs. Mathematically, given a request r and a TTL value τ , Continnum estimates $\text{Cost}(\tau, r)$ and $\text{Benefit}(r)$ for pinning the KV cache of request r for τ . For simplicity, $\text{Benefit}(r)$ assumes that the next request arrives within the TTL window. The case where TTL expires before the tool call returns is addressed in Sec. 4.2.

Cost Estimation. The cost of pinning a request’s KV cache comes from the opportunity cost of occupying GPU memory that could otherwise be used to serve other requests:

$$\text{Cost}(\tau, r) = \frac{\text{MemUsage}(r)}{\mathcal{M}} \times \tau,$$

where $\text{MemUsage}(r)$ is the amount of GPU memory used by the KV cache of request r , $\mathcal{M}$ is the average GPU memory footprint of active requests, and τ is the TTL value.

The ratio $\frac{\text{MemUsage}(r)}{\mathcal{M}}$ represents how many average requests are blocked when r is pinned. In other words, if pinning r occupies the same memory as k requests, then pinning r adds τ latency to approximately k other requests. We assume that the waiting queue always contains enough requests for this blocking effect to occur when KV retention is necessary.

Benefit Estimation. The benefit of pinning a request’s KV cache is realized when the request is re-issued within the TTL period, allowing it to avoid the overhead of reloading or prefilling the KV cache from r ’s program while saving the per-turn queueing delay:

$$\text{Benefit}(r) = \text{CacheMissCost}(r) + \text{OutOfOrderCost}(r)$$

Here, $\text{CacheMissCost}(r)$ measures the cost of reloading or prefilling the KV cache for request r and $\text{OutOfOrderCost}(r)$ measures the expected queueing delay for the request due to waiting for other requests to free GPU memory. We use the sum of cost prevented as the benefit.

Similar to $\text{Cost}(\tau, r)$, we can measure $\text{CacheMissCost}(r)$ by (1) the context reconstruct overhead $\text{Prefill-Reload}(r)$; and (2) the approximate number of requests will experience the additional latency

overhead $\frac{\text{MemUsage}(r)}{\mathcal{M}}$. The cost is formally defined as follows:

$$\text{CacheMissCost}(r) = \frac{\text{MemUsage}(r) \times \text{Prefill-Reload}(r)}{\mathcal{M}}$$

$\text{Prefill-Reload}(r)$ is the time cost for prefill or reloading depending on whether CPU offloading is turned on. This is based on a quick offline profiling described in Sec 5.2.

Measuring the expected queueing delay: As discussed in Sec. 3.2, retaining KV cache also eliminates the queueing delay that a returning program would experience if evicted—even when CPU offloading makes reload itself fast. This OutOfOrderCost component is the key term absent from prior retention policies such as InferCept [2], which only considers the reload cost. By modeling this term, Continnum can justify retaining KV cache even when reload is cheap, as long as the queueing delay savings outweigh the GPU memory occupation cost. Note that the queueing delay benefit is closely tied to the memoryfulness of the workload, *i.e.*, whether the number of remaining steps reduces predictably as the program progresses.

For example, if the number of requests issued by each program follows a geometric distribution, then the expected number of remaining requests is constant regardless of how many have already been served; in this case, pinning provides no benefit for the queueing delay since keeping the order does not accelerate finishing short jobs first. In contrast, if each program issues a fixed number of requests, then the TTL can eliminate the queueing cost by approximating Shortest Job First.

Let N be the total number of requests in a program and k the number of requests that have already been served. We define the following *memoryfulness factor*

$$\eta = -\text{Corr}(k, N - k)$$

We can see this factor models the degree of memoryfulness in the workload well: when the workload is fully memoryless, we have that k is independent to $N - k$, leading to $\eta = 0$. Conversely, when the workload is fully memoryful, *i.e.*, all programs have the same fixed number of requests, we have $\text{Corr}(k, N - k) = \text{Corr}(k, -k) = -1$, resulting in $\eta = 1$. Note that, in some cases η may be less than zero (extremely long-tail turn distribution), indicating an *anti-memoryful* pattern in which making progress on a program appears to reveal even more remaining work. We did not observe such patterns but Continnum is designed with such extreme workloads in mind: it would be preferable to serve each program only briefly and switch frequently to adapt to the long-tail turn distribution.

Now, we are ready to define the $\text{OutOfOrderCost}(r)$ based on the η above. When $\eta = 1$, the delay is exactly the waiting time when the program of r returns back to the waiting queue. To match this, we record the average waiting time per unit context size for the historical requests in this workload as $\frac{\mathcal{T}}{\mathcal{M}}$, where $\mathcal{T}$ is the average queueing delay for previous requests. In this case, the delay can be well measured by $\frac{\mathcal{T}}{\mathcal{M}} \times \text{MemUsage}(r)$. Here, we consider $\text{MemUsage}(r)$ since large-context requests are harder to schedule (they must wait for enough contiguous memory to be freed). For the general cases, we define the out-of-order cost as follows:

$$\text{OutOfOrderCost}(r) = \frac{\mathcal{T}}{\mathcal{M}} \times \text{MemUsage}(r) \times \eta.$$

4.2 Setting the TTL Value

In this part, we describe how Continnum sets the TTL value for KV cache based on the cost-benefit model above and historical tool-call information. As in Algorithm 1 (line 12), Continnum determines the optimal TTL value τ^* to maximize the expected net benefit of retaining the KV cache:

$$\tau^* = \operatorname{argmax}_{\tau} \mathcal{P}(\tau, f) \times \text{Benefit}(r) - \text{Cost}(\tau, r), \quad (1)$$

where $\mathcal{P}(\tau, f)$ estimates the probability that the tool call f completes within time τ . This formula captures the expected net benefit, in terms of total job latency, of retaining the KV cache of r for a duration of τ . By eliminating the shared $\frac{\text{MemUsage}(r)}{M}$, the formula above can be transformed to

$$\operatorname{argmax}_{\tau} \mathcal{P}(\tau, f) \times (\mathcal{T} \cdot \eta + \text{Prefill-Reload}(r)) - \tau, \quad (2)$$

indicating that we only need to additionally compute $\mathcal{T}$ and $\mathcal{P}(\tau, f)$ in our implementation. $\mathcal{T}$ can be estimated as the sliding window average for queueing delay experienced by requests who was evicted. Since we cannot fully predict the duration of the next tool call, we estimate $\mathcal{P}(\tau, f)$ using the empirical CDF derived from historical tool-call records $S[f]$. Specifically, we calculate it as the following:

$$\mathcal{P}(\tau, f) = \frac{1}{|S[f]|} \cdot \sum_{t \in S[f]} \mathbb{I}[t \leq \tau]$$

, where $\mathbb{I}[\cdot]$ is the indicator function. Finally, we solve Equation (2) by enumerating all unique tool-call durations recorded in $S[f]$ as candidates (including $\tau = 0$) and selecting the one with the highest expected reward.

Cold-start Handling. When the number of historical records in $S[f]$ is small, the empirical CDF estimation may be unreliable. In this case, we first try to use the global tool-call information to estimate $\mathcal{P}(\tau, f_{\text{any}})$, which can be computed as $\sum_{t \in S} \mathbb{I}[t \leq \tau] / |S|$.

Moreover, at the very beginning of engine serving, even the global records might not be reliable. To address this, we design a minimal version of Continnum that uses a fixed TTL threshold T_{default} , derived from the same cost model by assuming that the tool-call duration follows an exponential distribution with unit mean, i.e., $\text{ToolCallDuration} \sim \text{Exp}(1)$; and the workload is fully memoryful, i.e., $\eta = 1$. T_{default} is then set to the optimal τ^* under this scenario.

In practice, we set a threshold M to decide whether to use fixed TTL, global records, or the fine-grained estimation above based on $S[f]$. That is, we use T_{default} when $|S| \leq K$; otherwise, we use the global records when $|S[f]| \leq K$, and use the fine-grained TTL setting for the remaining cases. In our implementation, we set $K = 100$ and initialize $\mathcal{T}$ as zero.

Moreover, since agents are usually post-trained with the tools before production [12, 13, 38], users can also obtain these cost-model statistics during training.

4.3 Scheduling Priority

In order to keep the scheduling compatible with the TTL algorithm, we need to re-define the request priority in inference engines. Continnum introduces a TTL-aware priority that elevates pinned requests within TTL to preserve continuity while still preserving program-level FCFS ordering. Specifically, the scheduler assigns

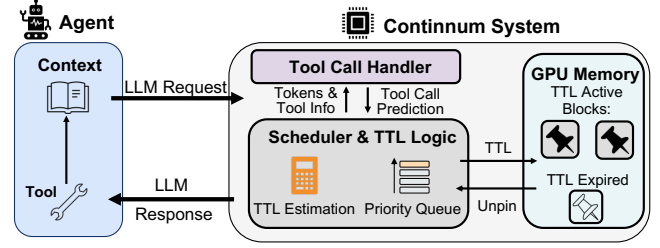


Figure 7: System Overview of Continnum

each request r in the waiting queue Q a multi-key priority tuple and ranks requests according to the following criteria (in order):

- **Preempted status:** Same as the original engine, preempted requests (due to running queue contention) are prioritized over non-preempted ones.
- **TTL status:** In other requests, requests retained within the TTL window are prioritized over unpinned ones.
- **Program-level arrival order:** Finally, within each category, requests are ordered by their program-level arrival time to maintain FCFS fairness.

5 CONTINNUM SYSTEM DESIGN

In Continnum, our design goal is a modular architecture that requires minimal changes to the core inference-engine scheduler loop. On the client side, we attach a program identifier (`program_id`) to every inference request so the system can recognize multi-turn agent programs and reason about tool calls across steps.

Upon arrival at the serving engine, requests enter the existing scheduler loop. Continnum adds a thin Tool-Call Handler that is invoked on request arrival and completion. The handler parses tool calls from LLM outputs, tracks per-tool latency using observed inter-request intervals within the same `program_id`, and returns TTL to the scheduler. The scheduler uses this hint to pin the request's KV cache for potential reuse by the next step, and later unpins it either when the TTL value expires or when the program terminates.

5.1 Tool Call Handler

The tool call handler is a separate class invoked by the main scheduler after the arrival or at the finish of a request. This decoupled structure ensures that tool handling logic remains isolated from the core scheduling loop, ensuring extensibility for future parsers or tool-aware policies.

Identifying the Tool Call: When the scheduler completes request, it forwards the response to the tool-call handler, which determines whether the response includes a tool invocation. The handler parses the message according to the function call schema, as the LLM outputs frequently adopt a standardized tool call structure such as the OpenAI schema:

```
{
  "id": "fc_0",
  "call_id": "call_0",
  "type": "function_call",
}
```

```

" name": "get_weather",
" arguments": {"location": "Paris"}
}

```

For this example schema, the handler checks each returned message block’s type; if it indicates a function/tool call, the handler extracts the call’s name and uses this as the tool call type. In SWE-Bench, it is guaranteed that each LLM’s response containing a function call will include exactly one bash function call. We extract the string within the bash block and use the first word afterwards as the tool call name.

More function call format examples for different LLMs [35, 44] can be handled by extending this parser with the same schema-specific extraction logic.

Recording the tool finish time: For each LLM request i in a program identified by a program ID p , the handler records a server-side completion timestamp $t_{\text{finish}}^{p,i}$ along with tool call name when scheduler records a finished request with tool call output. When the next request $i+1$ with the same p arrives, we observe its server-side arrival timestamp $t_{\text{arrive}}^{p,i+1}$ and compute the inter-request interval $t_{\text{arrive}}^{p,i+1} - t_{\text{finish}}^{p,i}$. We record this interval as the execution time of the tool call this time to store for TTL computation in the future.

5.2 Efficient Pin with TTL in Scheduler

After the tool call handler gives the TTL value, the scheduler will need to execute the pin operation.

Request Pinning: If the step is not signified to be the last step (ex. parsed to contain a tool call), the scheduler calls the tool-call handler to obtain the TTL value τ^* and, if not zero, invokes `pin_request(request,  $\tau^*$ )`. This records a pair of request and its expiration time `current_timestamp +  $\tau^*$` in a dictionary `pinned_requests` and deliberately skips freeing the request’s KV blocks. The `pinned_requests` dictionary will also be passed to the waiting queue to prioritize the scheduling of the next request in the same program.

Request Unpinning: At the beginning of every scheduling step, the scheduler runs `unpin_requests()`. It scans `pinned_requests` and unpins entries whose TTL have expired *and* whose `program_id` does not currently appear in the waiting queue. This prevents premature eviction when a follow-up request has already arrived at the inference engine but scheduler has not been able to schedule it. Additionally, when a program’s last step finishes, the scheduler proactively unpins any remaining pins with the same `program_id`, as no KV cache reuse is expected in the near future.

Prevention of deadlocks: Pinned requests can accumulate and potential deadlock could occur when all the GPU memory is occupied by the pinned requests. Since the pinned requests would be preserved if the next request of the same program is still in the waiting queue, the entire scheduling loop could be stuck and no new requests can be scheduled to run due to the lack of space.

Thus, we need a mechanism to unpin the requests when the such a deadlock occurs. In Continnum, when the scheduling logic fails to schedule a new request due to space contention, it will check if there are any pinned requests in `pinned_requests`. If there

are, we iteratively selects victims from `pinned_requests` with the latest program arrival time to unpin and free the space until the first request can be scheduled to run. The chosen request will be removed from its queue, its KV cache is freed, and it is re-queued as needed, ensuring that subsequent allocations can proceed to run. This prevents deadlock even when many pins are present.

Offline Profile: In order to predict the prefill time and reloading time (Prefill-Reload(r)) based on context size as needed in Sec 4.1, we perform an offline profile on each hardware and model pair for online estimation. We profile for two purposes: (1) GPU-CPU bandwidth for CPU offloading cases. We measure by taking the average CPU offloading throughput. (2) Prefill vs context length curve for estimating prefill cost. We measure this by doing prefill for chunk sizes $\{1000, 2000, 4000, \dots, \text{max_context_length}\}$ and fit a quadratic curve on the data. Admittedly, there could be some pages for the request remaining in GPU memory that does not need recomputation. But these remaining pages are usually small when memory is contended and we approximate by the full prefill time with little error. Profiling takes less than 10 minutes for each hardware model pair.

5.3 Implementation

We implemented Continnum on top of vLLM with about 1k lines of Python. Besides the above pinning operations added to the scheduler class, we use three functions from tool call handler in vLLM’s original scheduler:

- `func_call_finish(tool, timestamp)`: When a request finishes and is parsed to contain a tool call, this function informs the tool call handler to record the tool call starting time.
- `update_tool_call_time(program_id, timestamp)`: When a new request arrives, it denotes that the tool call from the previous request finished, so we record the time.
- `set_up_ttl(request, tool)`: Based on previous tool call information and the system setup, give the best TTL value for the scheduler for this finished request.

6 EVALUATION

Our key takeaways from the evaluation are:

- **Delay Reduction:** Continnum achieves significant delay reduction improvements over baseline schedulers through intelligent KV cache pinning
- **Robustness:** Continnum outperforms baselines across turn counts and different offloading scenarios.
- **Out-of-the-Box Usability:** Continnum can be used to run real agent faster without a quality drop.

6.1 Setup

Model and Hardware: We evaluate Continnum with Llama-3.1-8B, Llama-3.1-70B, and Gemma-3-12B. We use A100-SXM GPU from Runpod, H100 from AWS and Company A, and B200 GPU from on-prem servers.

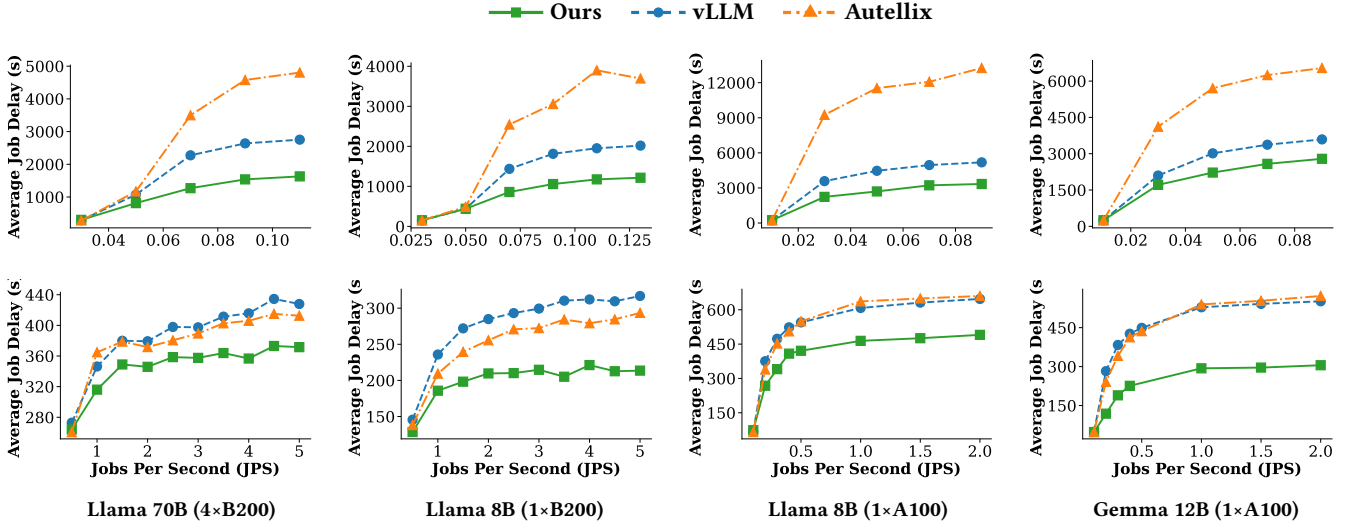


Figure 8: Continuum outperforms baseline schedulers across different model sizes, hardware configurations, and datasets (Top: SWE-bench; Bottom: BFCL).

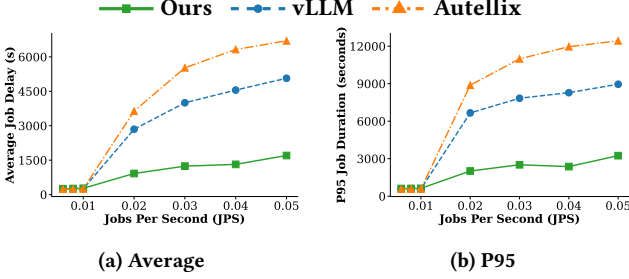


Figure 9: Continuum achieves best performance on OpenHands with Llama-8B on average and P95 delays with H100.

Datasets: For results other than the real SWE-Bench experiments in Figure 12, we evaluate on two collected workloads running GPT-5² and using poisson distribution for the arrival pattern of agent programs:

- SWE-Bench [24]: We run mini-swe-agent [33]³ on SWE-Bench. We keep requests within the context window.
- Berkeley Function Calling Leaderboard [40]: We used the latest version of BFCL V4 (Web Search category). This includes agents answering questions with web browsing tools. We scaled down the workload by 0.4 to fit at least 100 request in the context window of llama-3.1 (128k tokens).
- OpenHand [51]: OpenHands is a popular open-source coding agent. We run the multi-SWE-bench [64] example in the official repo for the Go language.

Main Baselines:

²We use GPT-5 for the better model capabilities to ensure that the workflow generated are mostly correct. Base small models often fail to accomplish the task

³SWE-bench official agent that rank #5 on leaderboard by Apr 13th

- *Vanilla vLLM* We use the stable release of vllm 0.10.2 with default setting, where chunk size is enabled with size 2048.
- *CPU DRAM offloading* We use vllm 0.10.2 with LMCACHE 0.3.7 [14]. For A100 GPUs, we set the DRAM size used in offloading to be 100GB; For B200 and H100 GPUs, we set the DRAM size used in offloading to be 200GB per GPU. We also apply this on top of algorithms below.
- *Autellix* We implemented the algorithm of PLAS from Autellix [39] on top of vllm. We extend Autellix to CPU offloading cases by enabling LMCACHE (Autellix+).
- *InferCept* We implemented the selectively preserve, swap, or evict algorithm of InferCept [2] on top of vllm + lmcache. Since the CPU offloading in LMCACHE is non-blocking (better than original InferCept), we update the cost estimation accordingly.
- *Distributed Inference* For real agent experiments, we compare with other open-source solutions including SGLang 0.5.5.post3 [47] with native cache-aware routing and Nvidia Dynamo 0.7.0.post1 [6] configured with 1P1D for PD disaggregation.

6.2 End-to-End Experiments

We conduct the trace replay experiments for SWE-Bench, BFCL, and OpenHands workloads. Figure 8, Figure 10, and Figure 9 demonstrate the end-to-end improvement of Continuum. We show significant improvements in both average response time and throughput across both the BFCL and SWE-Bench workloads. For instance, with the Llama-3.1-8B model, Continuum achieves up to a 2x reduction in average response time compared to the vanilla vLLM baseline. The performance gains are consistent across different model sizes and hardware configurations, demonstrating the effectiveness of our approach in diverse scenarios. Although Autellix outperforms baselines in BFCL, it underperforms in SWE-Bench due to its false assumption that requests have longer expected finish time if they execute for longer. Note that the job per second rates are less than

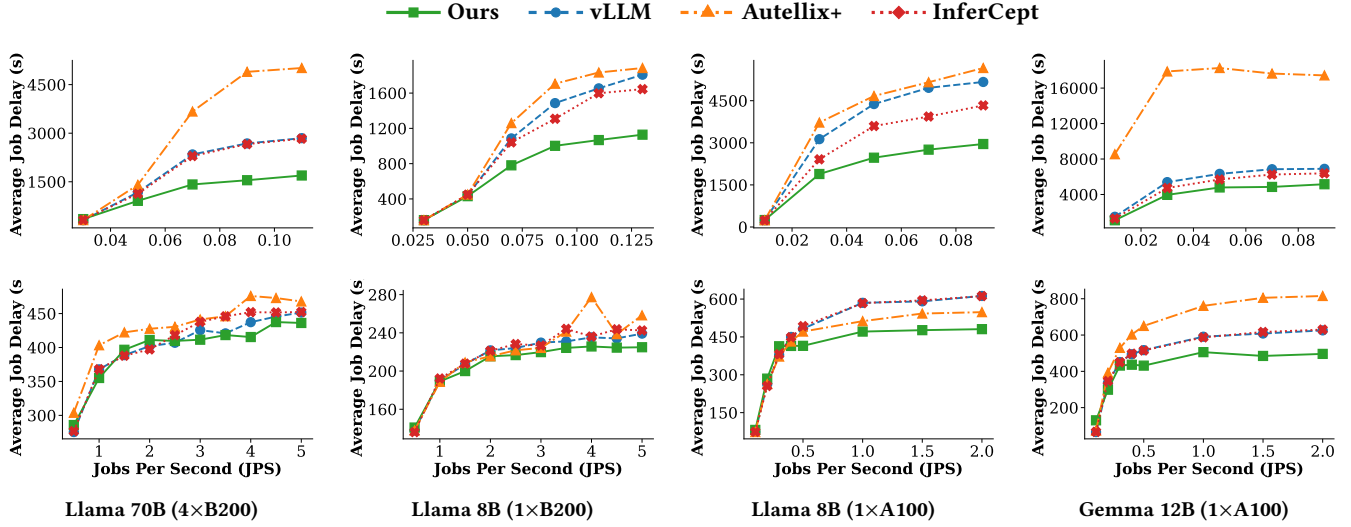


Figure 10: Continnum achieves consistent improvement when DRAM offloading is enabled. It improves over systems with smart DRAM offloading logic like InferCept by considering tool-call and multi-turn together (Top: SWE-Bench; Bottom: BFCL).

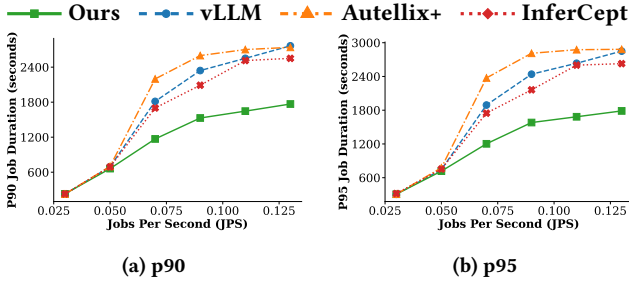


Figure 11: Continnum achieves better P90 and P95 latency for running SWE Bench trace with Llama-8B model.

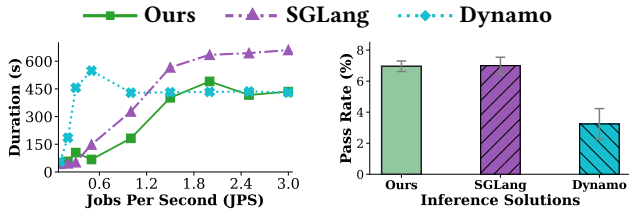


Figure 12: Continnum improves delay under the pass rate for real SWE-agents in distributed settings.

job per second reported in previous LLM serving papers. This is because agentic workloads are much more complex and can often involve more than 10 LLM inferences requests, incurring higher computational load.

We also extended our evaluation to other practical agents. As demonstrated in Figure 9, we achieve better delay running Open-Hands agent with Llama 8B on one H100 GPU from AWS. Since the average turn number count is higher, our improvement is even

more significant due to the deterioration of baselines under high turn numbers.

Moreover, we observe that Continnum consistently outperforms CPU offloading baselines. On the other hand, PLAS’s gain on CPU offloading diminished compared with baseline. This demonstrates Continnum’s robust performance improvement on scheduling bubble reduction that is orthogonal to DRAM offloading techniques.

In Figure 11, we show that Continnum achieves better P90 and P95 latency due to its ability to reduce the per-turn queueing delay compared with baselines. The setup for each individual point is running Llama-8B model with a single B200 with CPU offloading set as 200GB per GPU.

Real SWE-Agent in Distributed Setting: In order to fully evaluate Continnum’s performance in real-world deployment scenarios at scale. We test Continnum running a real SWE agent for 500 tasks in SWE-Bench-Verified in Company A’s internal H100 testbed. We set up our agent client environment by adding a job distributor for the SWE-Bench platform that distributes agents according to a Poisson distribution. We use a simple session-aware routing for Continnum and compare against other distributed inference solutions. We measure the per-job finish time and collect the pass rate of each agent program for their generated results on SWE-bench after generation.

As demonstrated by Figure 12, Continnum consistently outperforms baselines in terms of average delay when pass rates are equal. Notice that Continnum actually has higher pass rate than baselines. This is due to SWE-Bench’s time limit for environment Docker containers to prevent hanging. When the baseline’s running time exceeds 15 minutes it will be preempted and treated as a failure case. This proves Continnum’s usability in real production settings.

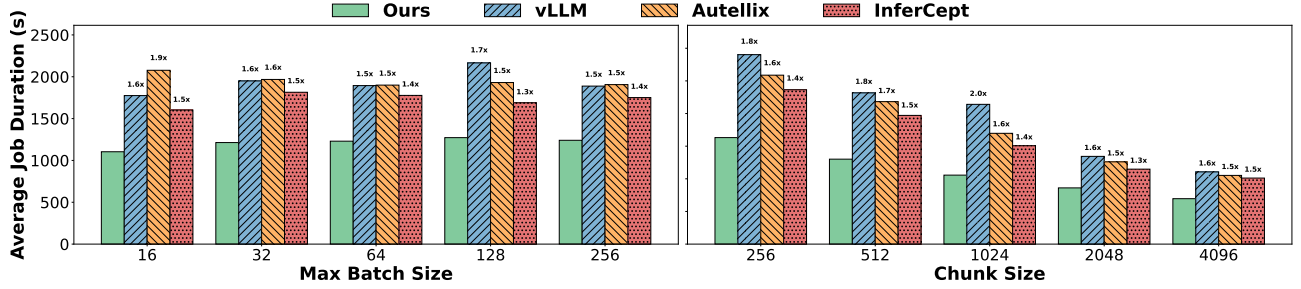


Figure 13: Continnum improves delay across different max batch-size and chunk-size configurations.

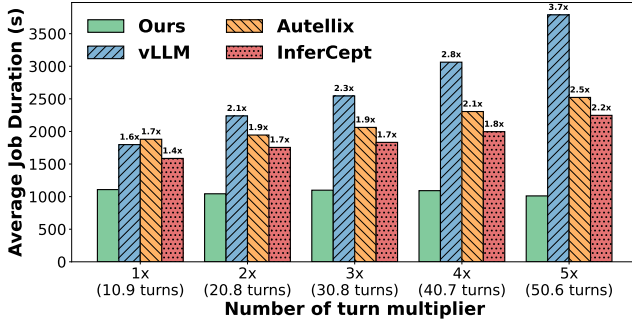


Figure 14: Continnum shows greater improvement as the number of turns increases, while the delay time remains stable.

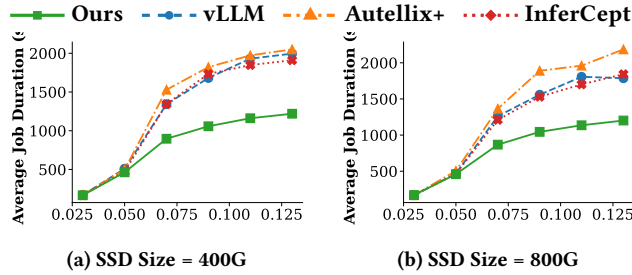


Figure 15: Continnum reduces delay when we extend offloading devices to SSDs beyond CPU offloading.

6.3 Sensitivity Analysis

Varying Inference Engine Configuration: In order to show that Continnum is robust to varying inference engine configurations, we evaluate Continnum with different configurations of the inference engine. In Figure 13, we set the job per second to be 0.13 and vary the maximum batch size to compare Continnum with different baselines. As we can see, Continnum’s improvement remains stable across different batch sizes. Moreover, in Figure 13, we vary the number of chunk size from 256 to 4096. We observe similar improvements across different chunk sizes. This demonstrates the robustness of our approach to different inference engine configurations.

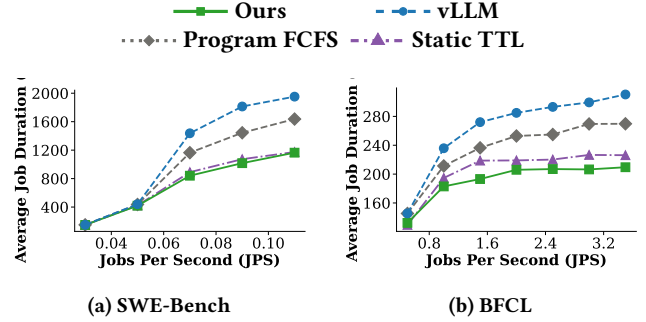


Figure 16: Contributions of individual ideas to Continnum. Program-level FCFS prioritizes requests with earlier program arrival instead of request arrival. Static TTL uses a fixed TTL threshold calculated from the cold-start handling mechanism.

Scaling Law for Turn Numbers: Figure 14 evaluates our scheduler’s robustness in multi-turn scenarios. We simulate more-turn scenarios on SWE-Bench by repeating the trace (1x to 5x) while inversely scaling the token lengths to emulate more turns but make total token fit within the context window. With a request rate of 0.13 JPS and 200 GB for DRAM offloading, the results show that the baseline methods degrade as the number of turns increases. This is because the increased number of turns leads to more tool calls and longer overall execution times, exacerbating the scheduling challenges faced by traditional methods. In contrast, our approach maintains stable, low-latency performance, demonstrating its effectiveness for complex, many-turn agentic interactions.

SSD Offloading: Similar to CPU offloading, SSD offloading offers bigger space but slower loading. We evaluate Continnum with extended SSD storage layer beyond CPU offloading using LMCACHE on SWE-bench workload with llama-8B on B200. As shown in Figure 15, Continnum consistently improves average delay compared with baselines when also utilizing disks of different sizes.

6.4 Ablation Studies and Microbenchmarking

Ablation Study: We conduct an ablation study to analyze the impact of our cost modeling on Continnum’s overall performance. In Figure 16, we compare Continnum with baselines that only

Table 4: Continnum introduces minor scheduling latency overhead comparison under different DRAM offloading settings.

System	No CPU Offload	CPU Offload
vLLM	0.95 ms	2.33 ms
Autellix	0.82 ms	2.18 ms
InferCept	N/A	2.25 ms
Ours	0.96 ms	2.30 ms

Table 5: Continnum achieves better performance on OpenHands rollout than concurrent work.

	vLLM	ThunderAgent	Continnum
Throughput (Steps Per Min)	93.4	114.8	144.9

applies part of the optimizations. Program-Level FCFS changes the original request-level FCFS in vLLM into priority based on program arrival. Static TTL builds upon program-level FCFS to utilize fixed TTL threshold estimated cold-start handling. As demonstrated, different ideas of Continnum gradually improves performance.

Scheduler Overhead: As shown in Table 4, our approach introduces a minor scheduling overhead compared to the baselines. However, this overhead is on the order of single-digit milliseconds, which is negligible compared to the GPU execution time for LLM inference. The significant end-to-end performance improvements from our scheduling strategy far outweigh this small increase in scheduling latency.

Application to Reinforcement Learning: We also conducted a micro-benchmark for potential reinforcement learning use of Continnum. We tested the OpenHands Agent with GLM-4.5-fp8 training on Multi-SWE bench [64] for rollout generation. The hardware setup is an 8xH100 node. We compared with the concurrent RL work ThunderAgent [26] on inference steps per minute, as reported by the original paper. As demonstrated by Table 5, Continnum achieves higher throughput for single node rollout.

7 RELATED WORK

LLM Inference Systems: There have been many research papers on improving LLM inference. Serving engines including vLLM [28] and SGLang [66] achieves state of the art inference by adapting paged attention design and optimized kernels. Besides the wide range of kernel-level optimizations that improve GPU execution speed [17, 62, 68], researchers have also proposed many optimizations on resource management: continuous batching [63], chunked prefill [5], skip-join multi-level scheduling [53]. Many of them have been ported into the inference engine. Previous work have also explored efficient offloading to CPU DRAM and disks [14, 18, 37, 55, 59]. For distributed inference, people have adopted session aware routing [29, 48], KV-cache aware routing [54], and prefill-decode disaggregation [67]. Building upon these work, Continnum extends LLM inference into long-horizon multi-turn agentic workloads and

improves resource management when resources are competed by different requests.

Time-to-live Mechanisms in Computer Systems: Time-to-live (TTL) is a longstanding abstraction in computer systems design, widely used in DNS resolvers, distributed caches, CDN edge nodes, and consistency protocols to bound staleness and prevent unbounded resource retention [11, 15, 21, 22, 25, 27, 32, 42, 43, 58]. In these settings, TTL acts as a coarse-grained validity window that balances freshness, load, and robustness under unpredictable update or fetch latencies. We build on this lineage but extend TTL to a new domain: fine-grained resource management inside LLM inference engines. Unlike traditional TTL uses, where entries are independent and correctness constraints are semantic rather than performance-critical, KV caches interact tightly with GPU memory pressure, prefill costs, and scheduling fairness in LLM serving engines. To our knowledge, Continnum is the first system to use TTL to regulate LLM KV cache as a function of predicted tool-call durations, scheduling-side delay propagation, and workload pattern.

Generality Beyond ReAct-Style Agents: The current design of Continnum are optimized for ReAct-style, tool-interleaving agents where each LLM step returns a clear tool invocation followed by a gap before the next step. Continnum naturally extends to parallel tool calls since it still follows the sequential “reason -> tool -> reason” rhythm. Some emerging agent frameworks, however, could involve non-linear control flows: speculative branches, asynchronous multi-agent coordination, and context folding. Although such workloads are mostly experimental and yet to be tested in real production workloads, their inference pattern may violate the sequential flow and requires future change. Extending Continnum to support such workloads is an important direction for future work.

8 CONCLUSION

Agentic workloads introduce new scheduling challenges for LLM serving systems due to frequent tool calls, highly variable inter-step delays, and the need to preserve multi-turn continuity. We present Continnum, a KV cache retention and scheduling system that balances both the benefit of cache reuse and the cost of blocking GPU memory through a time-to-live mechanism. By integrating TTL-based pinning with program-level FCFS, Continnum reduces unnecessary prefills, mitigates per-turn queueing delays, and robustly adapts to unpredictable tool-call latencies. Our implementation on top of vLLM shows consistent improvements in end-to-end job completion time across model sizes, hardware configurations, and real-world agent workloads. Continnum demonstrates that principled, tool-aware KV management is essential for efficient multi-turn agent serving. We hope it lays the groundwork for future systems to deeply integrate agent workload into LLM inference engines.

9 DATA AVAILABILITY

The implementation and artifacts are available at: <https://github.com/Hanchenli/vllm-continnum>

REFERENCES

- [1] 2025. Kimi K2 Tech Blog. <https://kimi-k2.org/blog>. Accessed: 2025-12-08.
- [2] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. INFERCEPT: Efficient Intercept Support for Augmented Large Language Model Inference. In *Forty-first International Conference on Machine Learning*. Vienna, Austria.
- [3] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925* (2025).
- [4] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.
- [5] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [6] ai dynamo. [n.d.]. Dynamo. <https://github.com/ai-dynamo/dynamo>. Accessed: 2025-12-09.
- [7] Anthropic. 2024. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>.
- [8] Anthropic / Claude. 2025. Claude Code. <https://claude.com/product/claude-code>. Accessed: 2025-12-11.
- [9] Anysphere. 2024. Cursor: The AI Code Editor. <https://cursor.com>.
- [10] Fengyao Bai, Hongbin Zhang, Zhitao Chen, Jiangsu Du, Zhiguang Chen, and Yutong Lu. 2026. AlignedServe: Orchestrating Prefix-aware Batching to Build a High-throughput and Computing-efficient LLM Serving System. *Proceedings of the ACM on Management of Data* 4, 3 (2026), 1–25.
- [11] Soumya Basu, Aditya Sundararajan, Javad Ghaderi, Sanjay Shakkottai, and Ramesh Sitaraman. 2018. Adaptive TTL-based caching for content delivery. *IEEE/ACM transactions on networking* 26, 3 (2018), 1063–1077.
- [12] Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R. Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. SkyRL-Agent: Efficient RL Training for Multi-turn LLM Agent. *arXiv:2511.16108* [cs.AI] <https://arxiv.org/abs/2511.16108>
- [13] Mingyue Cheng, Jie Ouyang, Shuo Yu, Ruiran Yan, Yucong Luo, Zirui Liu, Daoyu Wang, Qi Liu, and Enhong Chen. 2025. Agent-R1: Training Powerful LLM Agents with End-to-End Reinforcement Learning. *arXiv:2511.14460* [cs.CL] <https://arxiv.org/abs/2511.14460>
- [14] Yihua Cheng, Yuhua Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *arXiv preprint arXiv:2510.09665* (2025).
- [15] Edith Cohen, Eran Halperin, and Haim Kaplan. 2005. Performance aspects of distributed caches using TTL-based consistency. *Theoretical computer science* 331, 1 (2005), 73–96.
- [16] Cursor. 2025. Agents | Cursor. <https://cursor.com/agents>. Accessed: 2025-12-11.
- [17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv:2205.14135* [cs.LG] <https://arxiv.org/abs/2205.14135>
- [18] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Attentionstore: Cost-effective attention reuse across multi-turn conversations in large language model serving. *arXiv preprint arXiv:2403.19708* 52 (2024), 20–38.
- [19] Shihong Gao, Xin Zhang, Yanyan Shen, and Lei Chen. 2025. Apt-serve: Adaptive request scheduling on hybrid cache for scalable llm inference serving. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.
- [20] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. 2025. Pie: A Programmable Serving System for Emerging LLM Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) (SOSP '25). Association for Computing Machinery, New York, NY, USA, 415–430. <https://doi.org/10.1145/3731569.3764814>
- [21] Hendri Hendri, Rukmi Sari Hartati, Linawati Linawati, and Dewa Made Wiharta. 2024. Optimizing CDN Modeling with API Integration Using Time ToLive (TTL) Caching Technique. *Jurnal Ekonomi Manajemen Sistem Informasi (JEMSI)* 6, 2 (2024).
- [22] Tomas Hernandez-Quintanilla, Eduardo Magaña, Daniel Morató, and Mikel Izal. 2021. On the reduction of authoritative DNS cache timeouts: Detection and implications for user privacy. *Journal of Network and Computer Applications* 176 (2021), 102941.
- [23] Carlos E. Jimenez, John Yang, Kilian Lieret, Alex L. Zhang, and Ofir Press. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? <https://github.com/SWE-bench/SWE-bench>.
- [24] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. SWE-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).
- [25] Jaeyeon Jung, Arthur W Berger, and Hari Balakrishnan. 2003. Modeling TTL-based Internet caches. In *IEEE INFOCOM 2003. Twenty-second Annual Joint Conference of the IEEE Computer and Communications Societies (IEEE Cat. No. 03CH37428)*, Vol. 1. IEEE, 417–426.
- [26] Hao Kang, Ziyang Li, Xinyu Yang, Weili Xu, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. 2026. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. *arXiv:2602.13692* [cs.OS] <https://arxiv.org/abs/2602.13692>
- [27] Balachander Krishnamurthy, Craig Wills, and Yin Zhang. 2001. On the use and performance of content distribution networks. In *Proceedings of the 1st ACM SIGCOMM Workshop on Internet Measurement*. 169–182.
- [28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [29] LMCACHE Lab and vLLM. 2025. vLLM Production Stack. <https://docs.vllm.ai/p/rojects/production-stack/en/latest/>
- [30] LangChain. 2025. ReAct-style agents — LangChain documentation. https://python.langchain.com/api_reference/langchain/agents/langchain.agents.react.base.ReActChain.html.
- [31] LangGraph. 2025. StateGraph and graph-based state machines — LangGraph. https://langchain-ai.github.io/langgraph/concepts/agentic_concepts/.
- [32] David Lawrence, Warren Kumari, and Puneet Sood. 2020. Serving stale data to improve DNS resiliency. (No Title) (2020).
- [33] Kilian Lieret, John Yang, Carlos E. Jimenez, Alexander Wettig, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2025. mini-SWE-agent: The 100-line AI agent that resolves GitHub issues on SWE-bench. <https://github.com/SWE-agent/mini-swe-agent>.
- [34] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.
- [35] Tim Lin. 2025. Overview of Function Calling in Open-Source Models. <https://medium.com/%40c22647809/overview-of-function-calling-in-open-source-models-cc23e9b13360>.
- [36] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688* (2023).
- [37] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [38] Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alpav Ariyak, Colin Cai, Tarun Venkat, Shang Zhu, Ben Athiwaratkun, Manan Roongta, Ce Zhang, Li Erran Li, Raluca Ada Popa, Koushik Sen, and Ion Stoica. 2025. DeepSWE: Training a Fully Open-sourced, State-of-the-Art Coding Agent by Scaling RL. <https://www.together.ai/blog/deepswe>. Together AI blog post, July 2, 2025.
- [39] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025).
- [40] Huanzhi Mao, Raymond Tsao, Jingzhuo Zhou, Shishir G. Patil, and Joseph E. Gonzalez. 2025. BFCL V4: Web Search. https://gorilla.cs.berkeley.edu/blogs/15_bfcl_v4_web_search.html.
- [41] Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Jason Huang, Vishnu Suresh, Yixin Huang, Xiaowen Yu, Joseph E. Gonzalez, and Shishir G. Patil. 2024. BFCL V3 • Multi-Turn & Multi-Step Function Calling Evaluation. https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html.
- [42] Giovane CM Moura, John Heidemann, Ricardo de O Schmidt, and Wes Hardaker. 2019. Cache me if you can: Effects of DNS time-to-live. In *Proceedings of the Internet Measurement Conference*. 101–115.
- [43] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, et al. 2013. Scaling memcache at facebook. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. 385–398.
- [44] Qwen. 2024. Function Calling — Qwen Documentation. https://qwen.readthedocs.io/en/latest/framework/function_call.html.
- [45] Shuo Ren, Pu Jian, Zhenjiang Ren, Chunlin Leng, Can Xie, and Jiajun Zhang. 2025. Towards scientific intelligence: A survey of llm-based scientific agents. *arXiv preprint arXiv:2503.24047* (2025).
- [46] Keshav Santhanam, Deepti Raghavan, Muhammad Shahir Rahman, Thejas Venkatesh, Neha Kunjal, Pratiksha Thaker, Philip Levis, and Matei Zaharia. 2024. Alto: An efficient network orchestrator for compound ai systems. In *Proceedings of the 4th Workshop on Machine Learning and Systems*. 117–125.
- [47] sgl project. [n.d.]. sglang. <https://github.com/sgl-project/sglang>. Accessed: 2025-12-09.

- [48] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. 2024. Preble: Efficient Distributed Prompt Scheduling for LLM Serving. arXiv:2407.00023 [cs.DC] <https://arxiv.org/abs/2407.00023>
- [49] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1302–1316.
- [50] Noppanat Wadlorn, Junyi Shen, and Yao Lu. 2026. Efficient LLM serving for agentic workflows: A data systems perspective. *Proceedings of the ACM on Management of Data* 4, 3 (2026), 1–29.
- [51] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741 [cs.SE] <https://arxiv.org/abs/2407.16741>
- [52] Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. 2023. Mint: Evaluating llms in multi-turn interaction with tools and language feedback. *arXiv preprint arXiv:2309.10691* (2023).
- [53] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920* (2023).
- [54] Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker, and Ion Stoica. 2025. SkyWalker: A Locality-Aware Cross-Region Load Balancer for LLM Inference. arXiv:2505.24095 [cs.DC] <https://arxiv.org/abs/2505.24095>
- [55] Zhiqiang Xie. 2025. SGLang HiCache: Fast Hierarchical KV Caching with Your Favorite Storage Backends. <https://lmsys.org/blog/2025-09-10-sglang-hicache/>
- [56] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Ion Stoica, Joseph E. Gonzalez, Tianjun Zhang, and Shishir G. Patil. 2024. Berkeley Function-Calling Leaderboard. https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html
- [57] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793 [cs.SE] <https://arxiv.org/abs/2405.15793>
- [58] Juncheng Yang, Yao Yue, and KV Rashmi. 2021. A large-scale analysis of hundreds of in-memory key-value cache clusters at twitter. *ACM Transactions on Storage (TOS)* 17, 3 (2021), 1–35.
- [59] Jiayi Yao, Hanchen Li, Yuhua Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. 94–109.
- [60] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045 [cs.AI] <https://arxiv.org/abs/2406.12045>
- [61] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
- [62] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. arXiv:2501.01005 [cs.DC] <https://arxiv.org/abs/2501.01005>
- [63] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [64] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. arXiv:2504.02605 [cs.SE] <https://arxiv.org/abs/2504.02605>
- [65] Wei Zhang, Zhiyu Wu, Yi Mu, Banruo Liu, Myungjin Lee, and Fan Lai. 2025. Tempo: Application-aware LLM Serving with Mixed SLO Requirements. *arXiv preprint arXiv:2504.20068* (2025).
- [66] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
- [67] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [68] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Tian Tang, Qinyu Xu, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2025. NanoFlow: Towards Optimal Large Language Model Serving Throughput.

arXiv:2408.12757 [cs.DC] <https://arxiv.org/abs/2408.12757>